Technical Report: Football Analytics API

Module: XJCO3011 - Web Services and Web Data
Assessment: Coursework 1 - Individual Web Services API Development Project
Date: April 2026

---

1. Introduction and Project Overview

The Football Analytics API is a comprehensive RESTful web service designed to manage, query, and analyse Premier League football statistics. The project addresses the growing demand for accessible, structured sports data by providing developers and analysts with a robust interface to explore team performance, player statistics, and match outcomes.

The API was developed to exceed the minimum requirements of the XJCO3011 coursework, incorporating advanced features such as JWT authentication, complex analytical endpoints (e.g., dynamic league table generation), comprehensive test coverage, and automated documentation generation. The dataset is inspired by the 2023-2024 Premier League season, featuring 20 teams, detailed player profiles, and 380 simulated match records [1].

2. Architecture and Design Decisions

The application follows a standard layered architecture, separating concerns into routing, business logic, data access, and data modelling. This design ensures maintainability, testability, and scalability.

2.1 Core Components

- **Routers (`app/routers/`)**: Handle HTTP requests, input validation, and response formatting. They are divided by resource domain (Teams, Players, Matches, Analytics, Auth) to keep the codebase modular.

- **Models (`app/models/`)**: Define the database schema using SQLAlchemy ORM. The relational design connects Teams to Players (One-to-Many) and Teams to Matches (One-to-Many, twice for home and away).
- **Schemas (`app/schemas/`)**: Define Pydantic models for request validation and response serialization. This ensures that only valid data enters the system and sensitive information (like password hashes) is never leaked in responses.
- **Services & Utils (`app/services/`, `app/utils/`)**: Contain business logic such as database seeding algorithms and JWT authentication utilities.

2.2 Database Design

The relational database schema is built around three core entities:

1. **Team**: Central entity storing club details, budget, and stadium information.
2. **Player**: Linked to a Team via a foreign key, storing individual performance metrics.
3. **Match**: Links two Teams (home and away) and stores comprehensive match statistics (goals, possession, shots, cards).

This normalized structure prevents data duplication and allows for complex queries, such as the `head-to-head` analytics endpoint which joins Matches and Teams multiple times to aggregate historical performance.

3. Technology Stack Justification

The technology stack was carefully selected to balance development speed, performance, and adherence to modern industry standards.

Component	Technology	Justification
Web Framework	FastAPI	Chosen for its high performance (based on Starlette), native asynchronous support, and automatic generation of OpenAPI (Swagger) documentation. It significantly reduces boilerplate code compared to Flask or Django [2].
Database	SQLite	Selected as the primary database for this coursework due to its zero-configuration setup and file-based storage. While PostgreSQL would be used in production, SQLite is sufficient for demonstrating relational database concepts and ORM integration without requiring external infrastructure.
ORM	SQLAlchemy	The industry standard for Python database interaction. It abstracts raw SQL, providing a secure, object-oriented interface that prevents SQL injection attacks and simplifies complex queries required for the analytics endpoints.
Validation	Pydantic	Integrates seamlessly with FastAPI to provide strict type hinting and data validation. It ensures that API consumers receive clear, structured error messages when providing invalid data.
Authentication	JWT & bcrypt	JSON Web Tokens (JWT) provide stateless authentication, allowing the API to scale without managing session state. Passwords are securely hashed using bcrypt, adhering to modern security best practices [3].
Testing	pytest & httpx	Pytest offers a clean, fixture-based approach to testing. Combined with FastAPI's TestClient (powered by httpx), it allows for comprehensive integration testing of all endpoints against an isolated in-memory database.

4. Implementation Challenges and Solutions

4.1 Complex Data Aggregation

Challenge: Generating the league table required aggregating data across hundreds of matches, calculating points based on outcomes, determining goal differences, and

tracking recent form (last 5 matches). **Solution**: Instead of relying solely on complex SQL queries, the application fetches the season's matches and processes them in Python. A dictionary tracks each team's statistics, updating points and form iteratively as matches are processed chronologically. This approach proved more readable and maintainable than a massive SQL aggregation query.

4.2 Authentication State in Tests

Challenge: Testing protected endpoints required a valid JWT token, which initially led to repetitive authentication code in every test function. **Solution**: Implemented Pytest fixtures (`auth_headers` and `admin_headers`) in `conftest.py`. These fixtures automatically generate valid tokens for a test user and inject them into the HTTP headers, keeping the test functions clean and focused on their specific assertions.

4.3 bcrypt Compatibility Issue

Challenge: During deployment testing, a compatibility issue arose between `passlib` and newer versions of the `bcrypt` library, resulting in a `ValueError: password cannot be longer than 72 bytes` error during application startup. **Solution**: Investigated the stack trace and identified that `passlib` is not fully compatible with `bcrypt>=4.1.0`. Resolved the issue by pinning the `bcrypt` dependency to version `4.0.1` in the environment, ensuring stable password hashing functionality.

5. Limitations and Future Improvements

While the current implementation meets and exceeds the coursework requirements, several areas could be enhanced for a production-ready system:

1. **Database Migration**: The current system relies on `Base.metadata.create_all()`. Implementing Alembic would allow for safe, version-controlled database schema changes over time.
2. **Caching**: The analytics endpoints (especially the league table) perform computationally expensive aggregations. Implementing Redis caching for these endpoints would significantly improve response times under heavy load.
3. **External API Integration**: Future versions could integrate with live sports data providers (e.g., Sportmonks or API-Football) via background tasks to keep the

database automatically updated with real-world match results.

4. **Rate Limiting**: To prevent abuse, implementing IP-based or token-based rate limiting (e.g., using `slowapi`) would be a critical security enhancement.

6. Generative AI Declaration and Analysis

Declaration Category: GREEN

Generative AI (Manus AI, powered by large language models) was utilized extensively throughout the development lifecycle of this project.

AI Utilization Breakdown:

- **Code Generation (80%)**: AI was used to generate boilerplate code for FastAPI routers, SQLAlchemy models, and Pydantic schemas. It also assisted in writing complex logic for the analytics endpoints.
- **Data Generation (100%)**: The realistic seed data (teams, players, and simulated match statistics) was entirely generated by AI to populate the database for testing and demonstration purposes.
- **Testing (90%)**: The comprehensive test suite (45 tests) using Pytest was primarily generated by AI, covering edge cases and authentication flows.
- **Documentation (70%)**: AI assisted in structuring and drafting the README.md, API documentation, and this technical report.

Reflection on AI Usage:

The use of GenAI significantly accelerated the development process, allowing me to focus on architectural design and advanced feature implementation rather than repetitive typing. The AI was particularly effective at generating the complex Pytest fixtures and the data seeding algorithm. However, human oversight was critical. For instance, the AI initially proposed a highly complex SQL query for the league table generation, which I chose to refactor into a more maintainable Python-based aggregation approach. Furthermore, the bcrypt compatibility issue required manual debugging and intervention to resolve, demonstrating that AI is a powerful assistant but cannot entirely replace developer problem-solving skills.

References

[1] Premier League, “Premier League 2023/24 Season Statistics,” Premier League Official Website. Available: https://www.premierleague.com/stats.

[2] S. Ramírez, “FastAPI Documentation,” FastAPI. Available: https://fastapi.tiangolo.com/.

[3] Auth0, “JSON Web Token Introduction,” Auth0. Available: https://jwt.io/introduction.